FaaShare: Enabling Generic Low-Latency and SLO-Aware GPU Sharing in Serverless Computing

Zhuolong Jiang, Zinuo Cai, Yumou Liu, Ruhui Ma, and Rajkumar Buyya, *Fellow, IEEE*

***Abstract*—Serverless computing, also known as Function as a Service (FaaS), is an emerging cloud computing paradigm characterized by high scalability and low maintenance requirements, which has attracted significant attention from academia and industry. However, existing serverless platforms lack effective support for high-performance accelerators such as GPUs, preventing computationally intensive workloads from benefiting from this promising model. This can result in high latency response, failure of Service Level Objectives (SLO), and low GPU utilization for serverless functions. Consequently, inefficient utilization of high-performance accelerators has become a major bottleneck in the performance of serverless computing systems with GPU support. In this paper, we propose FaaShare, an efficient serverless computing platform that addresses three challenges: high latency, violation of SLO, and GPU sharing. First, we introduce a new container abstraction called MSContainer, which provides a pre-warm environment for function execution to reduce cold start latency. Second, we propose a temporal-spatio priority queue scheduling mechanism that dynamically adjusts function priorities to ensure service level objectives. Finally, we utilize Bayesian optimization to allocate optimal computing resources for serverless functions to enhance GPU sharing performance. Extensive evaluation proves that the proposed framework substantially outperforms state-of-the art systems by 23%, 33%, and 18% on job completion time, makespan, and system throughput, meeting the low latency goals of serverless services.**

***Index Terms*—Serverless computing, cloud computing, GPU sharing.**

I. Introduction

SERVERLESS computing [1], [2], [3], [4], [5], [6], [7] is rapidly emerging as a promising paradigm for the next generation of cloud computing. Compared with existing cloud paradigms, including Infrastructure as a Service (IaaS) [8], Platform as a Service (PaaS) [9], serverless computing, known as Function as a Service (FaaS), takes function as the minimum execution unit, providing a more lightweight resource abstraction from the perspective of developers. For cloud vendors, serverless computing can improve resource utilization by fully controlling the scheduling of functions and promoting their cloud services for conveniences, such as object storage and logging tools. Due to the enormous potential of serverless in lightweight parallelism, academia and industry also utilize serverless platforms to deploy computation intensive applications like large language models (LLMs) [10],distributed model training [11], [12], [13], [14], [15], serving [16], [17], [18], [19], video processing [20] and scientific computing [21].

To facilitate the GPU usage in serverless computing, existing methods have made efforts, but there are still limitations. For the high latency of GPU usage in serverless, INFless [16] reduces the cold start of functions at the container level. DGSF [22] reduces latency at the process level by preloading the CUDA runtime to decrease the execution time of the function's initial invocation, but it overlooks the loading time of other third-party libraries and the container restart time due to program errors. From the perspective of service level objectives (SLO) guarantee, FaaSwap [23] proposes an SLO-aware request queueing policy, but it is only used for model inference. To improve GPU utilization in serverless, ElasticFlow [24] specifically considers deep learning training scenarios in serverless and improves GPU utilization by dynamically allocating resources based on task requirements. However, the unit of resource allocation remains a single GPU, which results in low GPU utilization. Although FaST-GShare [25] allows GPU sharing for serverless functions in inference scenarios by adopting Multi-Process Service (MPS)[1], it does not take into account resource contention among tasks. Thus, it is necessary to design a low latency and SLO-aware GPU sharing mechanism for GPU-based high-performance computing tasks in serverless.

However, it is not trivial to enable low latency and SLO-aware GPU sharing in serverless computing. We implement a naive GPU-enabled serverless computing platform based on Apache OpenWhisk[2] as our baseline and identify three critical challenges. The first challenge is the high latency compared with the prompt execution time improved by high-performance accelerators, especially for latency-sensitive workloads like online inference [19] and real-time data processing [26]. Our experiments in II-B reveal that cold start consists of two components, CUDA context initialization, library loading latency e.g., cuDNN, PyTorch, TensorFlow. The components can be four times longer

Received 9 September 2024; revised 3 June 2026; accepted 10 June 2026. Date of publication 16 June 2026; date of current version 6 July 2026. This work was supported by the Shanghai Municipal Science and Technology Major Project. Recommended for acceptance by R. Tolosana. *(Zhuolong Jiang and Zinuo Cai contributed equally to this work.) (Corresponding author: Ruhui Ma.)*

Zhuolong Jiang, Zinuo Cai, Yumou Liu, and Ruhui Ma are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: jzz19961996@sjtu.edu.cn; kingczn1314@sjtu.edu.cn; liiyumou@sjtu.edu.cn; ruhuima@sjtu.edu.cn).

Rajkumar Buyya is with the Quantum Cloud Computing and Distributed Systems Laboratory, School of Computing and Information Systems, University of Melbourne, Parkville, VIC 3010, Australia (e-mail: rbuyya@unimelb.edu.au).

Digital Object Identifier 10.1109/TPDS.2026.3704608

[1]https://docs.nvidia.com/deploy/mps/index.html

[2]https://openwhisk.apache.org/

1045-9219 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

than the execution time of CUDA applications and 60× longer than that of the inference latency for deep learning models.

The second challenge is the violation of SLO. Co-location affects the execution of each task, which may lead to exceeding the time limits set by users for their tasks. Thus, it is crucial to ensure that each function meets its time constraints under co-location. To improve SLO performance in co-location, FaST-GShare [25] divides all functions into two groups based on the likelihood of meeting SLO. It maintains queues with different priorities for each group. However, it overlooks the relationship between memory usage and the execution order of different tasks in co-location under different workloads.

The third challenge is enabling efficient GPU sharing to support the co-location of serverless functions. As the number of concurrent requests increases, more tasks need to be processed simultaneously in the GPU, which can lead to complex resource allocation and performance degradation. Therefore, allocating the optimal GPU resource configuration and reducing resource contention for each function is crucial. NVIDIA provides MPS to allow multiple processes' CUDA kernels to run concurrently on the same GPU, improving GPU concurrency and utilization, but it does not consider optimal resource configuration for co-location. Degradation-aware solution [27] models and predicts the degradation of co-location to configure GPU resources, but it ignores the computing resource competition. Heuristic-based solution [25] identifies the configuration based on sampling function resource quotas, but it ignores the over-subscription of GPU utilization and lacks flexibility in resource configuration.

To overcome these challenges, we propose FAASHARE, an efficient GPU-enabled serverless framework designed to address three goals: low latency, satisfaction of SLO, and high GPU utilization. For low latency, FAASHARE provides pre-warming mechanisms for serverless functions, significantly reducing cold start delays during initial execution. To guarantee SLO, FAASHARE offers dynamic priority adjustments for different workloads in over-subscribed scenarios, ensuring fair execution opportunities for each task to meet co-location SLO requirements. To improve GPU sharing, FAASHARE utilizes a Bayesian Optimization-based (BO-based) GPU sharing strategy to allocate the optimal computing resource configurations for co-location and reduce resource contention.

To be specific, we firstly propose MSContainer, a novel container abstraction to achieve low latency for GPU-required serverless functions. Initially, MSContainer creates Prewarmer to offer warm-up environments for serverless functions, then injects the functions into the Prewarmer and monitors their entire lifecycle. MSContainer backs up the Prewarmer's state before each function execution, enabling the program to recover from abnormal exits. To demonstrate the universality of cold start optimization in MSContainer, we further extend its support to other high-level programming languages, like Python and Java.

To ensure SLO guarantee, we propose a temporal-spatial priority queue scheduling mechanism. It predicts task degradation and provides fair execution opportunities for different types of functions in over-subscribed scenarios. It considers both the memory footprint and execution time of tasks, which not only enhances memory utilization but also ensures tasks meet their time constraints. It offers a real-time priority update strategy for task requests, iteratively adjusting the execution order of the task queue to optimize task performance. Moreover, the priority update strategy ensures that the order of execution is not influenced by the order of requests, allowing each task to have a fair chance of execution.

To address the challenge of GPU sharing, we introduce BO and combine it with MPS to propose a serverless-specific BO-based GPU sharing strategy to allocate optimal computing resources for serverless functions to enhance GPU sharing performance. Compared to traditional BO, our approach offers three optimizations. First, it dynamically adjusts the granularity of resource allocation based on the characteristics of tasks in co-location to adapt to varying workloads. Second, it modifies the preferences for exploration and exploitation in BO to expedite the search for the optimal configuration for co-location. Third, it dynamically updates based on existing configurations to generate new resource allocations rapidly to adapt to resource optimization for new tasks swiftly.

To demonstrate the effectiveness of our serverless framework for the abovementioned challenges, we build FAASHARE, a low latency, SLO guarantee, and high GPU utilization serverless framework. We conduct extensive evaluation to prove that FAASHARE can substantially outperform state-of-the art systems by 23%, 33%, and 18% on job completion time, makespan, and system throughput, while meeting the low latency goals of serverless services.

Our contributions are as follows:

- We propose FAASHARE, a low latency and efficient SLO-aware GPU sharing serverless framework to enable low latency, SLO guarantee, high GPU utilization for serverless function.
- We design an MSContainer, a novel container abstraction to provide a pre-warm environment for serverless functions for low latency challenge to reduce cold start latency.
- We design a temporal-spatio priority queue scheduling to predict task degradation and provide dynamic priority and fair opportunity in oversubscribed scenarios to ensure SLO for serverless function.
- We propose a BO-based GPU sharing approach to allocate the optimal computing resource configuration and mitigate resource competition for co-location to improve GPU utilization.
- We implement FAASHARE based on OpenWhisk and demonstrate its significant improvement in cold start, SLO guarantee, GPU sharing through extensive evaluation.

II. Background and Motivation

A. Serverless Computing

Serverless computing, also known as Function as FaaS, is an emerging paradigm in the field of cloud computing, and it is rapidly growing in popularity. The traditional IaaS model typically revolves around servers, where users are responsible for managing hardware resources to meet specific workload demands. However, serverless computing completely changes this model by outsourcing the responsibility of resource management

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

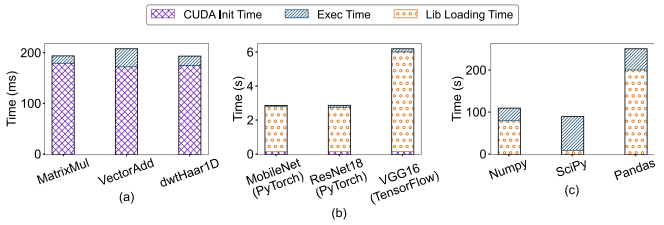


Fig. 1. Runtime analysis of CUDA-based programs.

to cloud service providers. Users only need to program without considering the underlying servers or resource configurations, simplifying code development. Serverless computing also offers high scalability, allowing for dynamic resource allocation based on workload demands, leading to more efficient resource utilization and cost control. These advantages present new prospects for tasks such as deep learning inference [16], [22], [25] and training [24].

To provide high-performance hardware support for computationally intensive tasks, we implement a naive version of GPU-enabled serverless platforms based on Apache OpenWhisk as our baseline, which can support GPU allocation according to the function resource requirements. By modifying the core components of Controller, and Invoker, our baseline can support GPU allocation according to the function resource requirements and memory and CPU provided by typical serverless frameworks. We deploy the GPU-enabled OpenWhisk in a distributed Kubernetes cluster[3], a production-grade container orchestration system and evaluate the baseline with CUDA applications and real-world workloads.

B. Low Latency Challenge

Cold start poses a critical challenge in serverless computing, particularly exacerbating scenarios with GPUs featuring lazy initialization characteristics. Cold start is typically composed of CUDA initialization and library loading, which is intolerable, especially for latency-sensitive workloads like deep learning inference tasks. To demonstrate the effect of cold start on the task, we evaluate cold start and pre-warm performance at baseline. Fig. 1 shows the response latency of GPU cold start when handling, three high-level model serving applications and three dependency-library loading workloads, NumPy, SciPy, and Pandas. We evaluate three pure CUDA applications, Matrix Multiplication (MM), Vector Addition (VA), and DwtHaar1D (DH), and three high-level model serving applications. Our evaluation shows that the cold start latency can be at most $120\times$ the actual execution time, which is highly unacceptable for latency-sensitive workloads. Library loading occurs before CUDA initialization and mainly includes various acceleration libraries for scientific computing, such as PyTorch, TensorFlow, cuDNN, etc. Although these libraries have different latencies, they significantly impact the execution time of the job. Moreover, CUDA initialization extends the delay time for CUDA-based applications in Fig. 1 (a).

[3]https://kubernetes.io/

TABLE I
WORKLOADS USED TO EVALUATE FAASHARE

Type	Name	Configuration	SLO (ms)
Native CUDA	MatrixMul [a]	51,1024,2048,4096	-
	VectorAdd[a]	10^5,10^6,10^7,10^8	-
	dwtHaar1D[a]	64k,128k,256k,512k	-
	Image encode and decode[a]	1024, 2048,4096, 8192	10,50,280, 600
	PCA [28]	20k, 40k, 80k	-
	KNN [29]	32, 64,128, 256	-
	BM3D [30]	256,512,1024, 2048	40,110,400,1600
	Video Process[b]	720p, 1080p, 4k, 8k	-
DL	ResNet-152 [31]	16, 32, 64, 128	108, 207, 389, 751
	ResNet-101 [31]	32, 64, 128, 256	145, 273, 525, 1020
	ResNet-50 [31]	16, 32, 64, 128	46, 87, 164, 317
	ResNet-34 [31]	64, 128, 256, 512	94, 180, 360, 705
	ResNet-18 [31]	8, 16, 32, 64, 128	9, 15, 29, 54, 102
	DenseNet-121 [32]	16, 32, 64, 128	52, 96, 180, 350
	DenseNet-161 [32]	32, 64, 128, 256	219, 407, 780, 1550
	DenseNet-201 [32]	16, 32, 64, 128	89, 158, 293, 566
	VGG-19 [33]	16, 32, 64, 128	107, 209, 410, 806
	VGG-16 [33]	8, 16, 32, 64, 128	47, 89, 172, 338, 667
	MobileNet [34]	64, 128, 256, 512	60, 117, 232, 459
	Qwen2.5-7B [35]	1	200
	Llama-3.2-3B [36]	1	160
	DeepSeek-R1-1.5B [37]	1	140

[a] https://developer.nvidia.com/cuda-downloads
[b] https://github.com/NVIDIA/VideoProcessingFramework

Challenge I

GPU cold start results in longer completion times for serverless tasks using GPU devices and is primarily composed of pre-warm window, CUDA initialization and library loading.

C. SLO Challenge

SLO guarantee is key to the serverless paradigm to enhance user experience. We first analyze the characteristics of popular GPU-based tasks, and then randomly select different numbers of tasks from Table I to describe the impact of priority execution strategies on performance.

Fig. 3 shows the task distribution for different model inference, model training, and native CUDA applications, "-T" represents training task, and reveals four typical performance and resource utilization types. MobileNet and VGG-16 exhibit low memory needs and rapid execution. In contrast, DenseNet-201 training shows high memory use but reasonable execution times. KNN tasks use less memory but are computationally demanding. Lastly, some tasks have high memory demands but brief execution times.

To schedule complex tasks in serverless, we perform repeatable random sampling of tasks from Table I and employ default, time-priority, and memory-priority scheduling strategies, with outcomes depicted in Fig. 2. Time-priority sorts tasks based on their execution time from shortest to longest, memory-priority

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

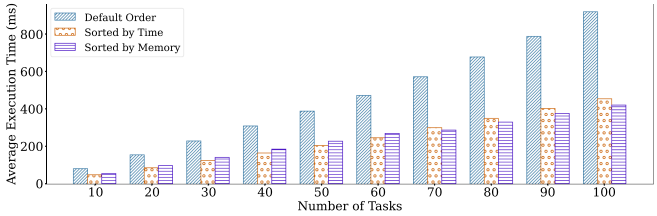


Fig. 2. Performance comparison of different priority strategy.

sorts tasks from smallest to largest based on memory usage, and default executes tasks in a random order. Both time-priority and memory-priority outperform the default, highlighting their importance in enhancing task performance. For instance, time-priority reduces average execution time in scenarios like 60 tasks, while memory-priority excels in others, such as with 100 tasks, showing that time and memory factors should be considered together in scheduling. The results reveal that without optimizing task order, significant performance degradation occurs. While time and memory strategies offer advantages over the default, they do not always ensure optimal execution.

This indicates that task scheduling can lead to significant performance degradation without optimizing the task order. Although the time-priority and memory-priority strategies have significant advantages over the default strategy, it does not provide the optimal execution order.

Challenge II

The co-location of model training, model inference, and native CUDA applications simultaneously impacts the SLO.

D. GPU Sharing Challenge

We employ the official device plugin in the naïve implementation to support GPU scheduling in large clusters, but its exclusive scheduling pattern does not allow GPU sharing among different serverless functions, which results in low device utilization in the naïve implementation. Fortunately, NVIDIA officially provides MPS to support fine-grained GPU sharing. The MPS mechanism allows parallel execution without context switching, significantly improving GPU efficiency from the software level. However, it cannot allocate suitable configurations for different tasks and alleviate resource competition in co-location.

Furthermore, Fig. 4 illustrates the system throughput slowdown comparison across three workload combinations under co-location. In memory-intensive scenarios, contention for memory bandwidth leads to performance degradations of 37% and 41%, respectively. For mixed workloads, ResNet-152 suffers a 60% throughput drop, while Qwen2.5-7B experiences a 30% reduction. This demonstrates that memory-intensive jobs tend to preempt resources and subsequently suppress compute-intensive jobs. Similarly, the compute-intensive combination results in throughput drops of 70% and 55% due to SM resource saturation. These results confirm that diverse workload combinations induce significant performance degradation, highlighting the necessity of optimizing resource configurations in GPU sharing scenarios.

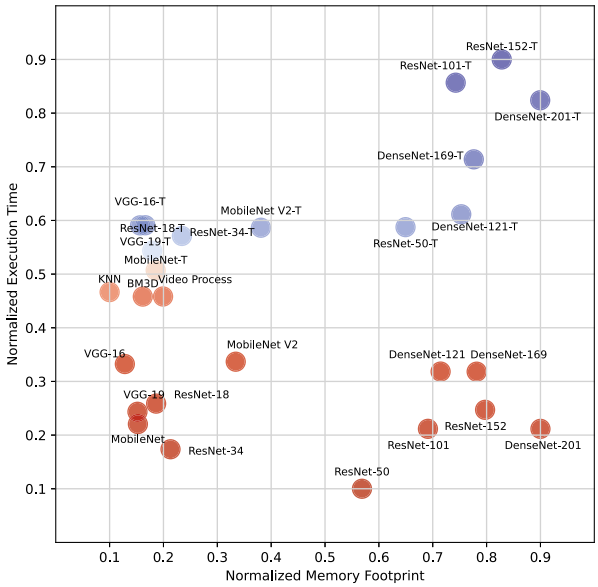


Fig. 3. Distribution of GPU-based workloads.

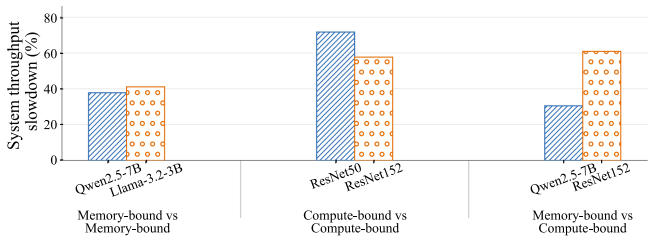


Fig. 4. System throughput slowdown comparison of workloads in MPS mode.

Challenge III

MPS can provide parallel capabilities for serverless function, but the challenges lie in finding the optimal configuration and reducing resource contention while improving GPU utilization for co-execution serverless workloads.

III. System Design

We propose FaaShare, an efficient GPU-enabled serverless framework to achieve low latency, satisfaction of SLO, high GPU utilization. Fig. 5 shows an overview of the system. FaaShare has four core components: an MSContainer, a Controller, a Scheduler, and an Optimizer. First, we employ MSContainer to provide a pre-warm environment for functions, mitigating cold start latency and enabling function execution on GPUs. Second, the Scheduler performs queue-level decisions by determining task ordering and candidate co-location sets according to SLO urgency and memory constraints, so that latency-critical tasks can obtain timely execution opportunities. Third, the Optimizer performs resource-level decisions for the tasks selected by the Scheduler. Specifically, it refines the MPS

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

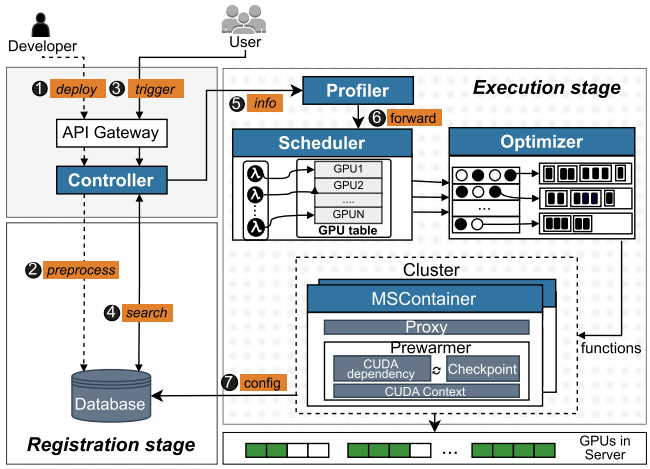


Fig. 5. System overview.

quotas of tasks within the selected co-location set, thereby improving GPU utilization while preserving SLO feasibility. In this design, the Scheduler decides which tasks run together and in what order, while the Optimizer decides how much GPU share each selected task receives. Finally, the Optimizer can allocate optimal computing resources for different workloads and task characteristics to achieve the best co-location performance.

Workflow of FAASHARE is similar to the function lifecycle mechanism on existing serverless platforms, and partitions the workflow into two stages: function registration and execution. In the registration stage, developers first deploy their functions that require GPU resources during execution to the serverless platform ❶. The Controller accepts the request, performs preprocessing ❷. Then the Controller stores the code file in the Database.

In the execution stage, after users trigger function invocation requests to the serverless platform ❸, the Controller retrieves the code file and searches for a suitable configuration in the Database based on the request information and current GPU load information ❹. If a suitable configuration is found, the Controller informs the Profiler ❺to forward the requests to the Scheduler ❻. Otherwise, the Profiler notifies the Optimizer to retrieve the nearest historical configuration as the initial MPS configuration and triggers asynchronous BO updates to refine the configuration table for future similar requests ❼. Then, the Controller forwards the requests to the Scheduler. At the same time, the Profiler is also responsible for monitoring the status and data in the GPU in real-time, including memory usage, task execution time, and storing the data in the Database. Finally, the Scheduler arranges fair sharing opportunities and sends the request to the MSContainer for execution to reduce cold start latency.

IV. MSCONTAINER

In this section, we propose MSContainer, a novel container abstraction designed to reduce cold start latency for serverless functions since we observe that cold start is a bottleneck when enabling GPU support for serverless platforms in Section II-B, especially for latency-sensitive workloads. We conduct the first

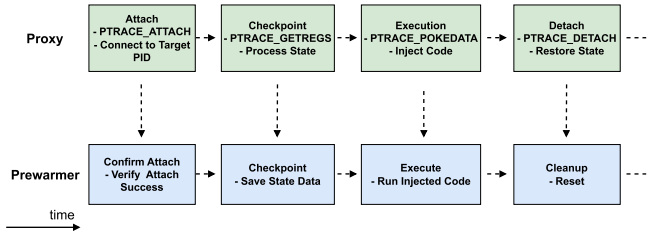


Fig. 6. The process of dynamic injection.

trial to design an application-agnostic mechanism to enable fast application startup. The core insight of MSContainer is pre-warm beforehand to alleviate shared context initialization latency. The key components of MSContainer are the cold start management, dynamic injection, multi-programming language extension. Cold start management is used to manage the creation and destruction of a Prewarmer that contains a pre-warm context. Dynamic injection allows serverless functions to be attached to the Prewarmer as plugins, monitoring and interacting with the Prewarmer. Multi-programming language extension makes cold start optimization generally support other high-level programming languages.

A. Dynamic Injection

We employ a dynamic injection mechanism to enable serverless functions to access Prewarmer to reduce cold start latency. The core of the dynamic injection mechanism is `ptrace`. `ptrace` is a system call that enables the parent process to observe and control the status and execution of its child processes. We create a Proxy to leverage `ptrace` to inject function into Prewarmer. Fig. 6 shows dynamic injection primarily consists of four stages: attach, backup, execution, detach.

Attach: The main purpose of the Attach stage is to obtain the runtime context of the Prewarmer, including memory layout, thread information, variable states, etc. The Proxy utilizes `ptrace` to establish a communication channel with the Prewarmer, enabling interaction in subsequent stages. This includes sending instructions to the target program, retrieving execution results, or receiving updates on the program's status.

Checkpoint: The checkpoint stage involves copying or extracting the memory, register state, files, and other critical data of the target program. Its purpose is to restore the original state of the target program when performing injection operations, if necessary. Because injection operations may modify the Prewarmer's memory layout, register state, and data structures. By creating a copy of the target program during the backup stage using `ptrace`, it is possible to restore it to its original state in case of problems during the execution phase.

Execution: In the execution stage, the Proxy injects a serverless function into the Prewarmer and executes the function. This is achieved by modifying the execution flow of the Prewarmer to redirect it to the entry point of the injected code. Specifically, the Proxy first utilizes `ptrace` to create a memory space within the Prewarmer. Then, using the memory writing capability provided by `ptrace` copy the function into the previously allocated memory region. Finally, the Proxy utilizes `ptrace` to modify

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

the program counter of the Prewarmer to point to the address where the function is executed.

Detach: In the detach stage, the Proxy unloads the serverless function from the Prewarmer and cleans up the associated resources, restoring the Prewarmer to its original state before injection.

B. Multi-Programming Language Extension

To demonstrate the generality of our approach, we further elucidate the cold start optimization principles in Python and Java.

The core of implementing dynamic injection in Python lies in the interoperability between Python and C. First, the code of the serverless function is compiled into a dynamic link library (DLL). Then, a Proxy, which is used for Python injection, integrates the DLL into the runtime of the Prewarmer using the `ctypes` or `cffi` modules. Finally, the injection logic is implemented for Python based on C, to perform the attach, checkpoint, execution, and detach operations on the Prewarmer, completing the entire dynamic injection process. This migration of dynamic injection technology from C to Python reduces cold start time. Thus, dynamic injection is enabled to transfer to Python to reduce cold start. For Java, dynamic injection utilizes the Java Virtual Machine's (JVM) capabilities to manipulate and extend the runtime behavior of Java applications. First, we initialize a specialized Java process as a Prewarmer. Then, a Java Proxy is created to package the serverless function into a JAR file and perform dynamic injection logic. The Proxy utilizes the JVM's API to implement injection logic to execute the injection process into the Prewarmer. This method allows for the dynamic injection of code directly into the Java runtime, significantly reducing cold start and improving execution efficiency.

C. Multi-Layer Defense Mechanism

To mitigate the security risks and potential instability inherent in dynamic injection, FAASHARE incorporates a comprehensive multi-layer defense mechanism. This framework spans the entire lifecycle of the injection process, including pre-injection handshakes, code integrity verification, runtime sandboxing, and fault recovery.

Safe-point Handshake: To eliminate risks such as deadlocks or context corruption caused by asynchronous process interruption, we introduce a synchronization mechanism. Specifically, the Prewarmer sets a `Ready` flag upon completing its initialization. The Proxy is permitted to execute `PTRACE_ATTACH` only when the `Ready` flag is asserted, ensuring that injection operations do not interfere with the Prewarmer's critical sections, such as memory allocation or locking operations.

Integrity Verification and $W \oplus X$ Protection: To safeguard against tampering and overflow attacks, the Proxy performs a SHA-256 checksum verification prior to injection. Furthermore, we adhere to the $W \oplus X$ (Write XOR Execute) principle: memory permissions are transitioned atomically from writable to executable only after the code has passed integrity verification and has been successfully committed to memory.

TABLE II
DESCRIPTIONS OF THE INPUT FEATURES FOR THE PREDICTOR

Name	Description
Task features	SM utilization, memory bandwidth utilization, and L2 cache utilization when the task runs in isolation.
Contention features	Aggregate SM utilization, memory bandwidth utilization, and L2 cache utilization of the other co-located tasks on the same GPU.
MPS configuration	Assigned MPS percentage limit for the task.

Post-injection Sandboxing: To prevent user functions from leveraging the injection channel for privilege escalation (Security), we enforce strict system call filtering. The Prewarmer installs a `seccomp-bpf` filter before executing user code, which intercepts and denies dangerous system calls (e.g., `ptrace` itself).

This mechanism effectively constrains the execution environment and restricts any malicious logic within the user code.

Atomic Rollback Mechanism: To prevent process crashes resulting from injection failures, the Proxy snapshots all register states of the Prewarmer before any memory modification occurs. If the injection process triggers an exception (e.g., a segmentation fault), the Proxy immediately intercepts the signal and leverages the cached register data to force-reset the CPU registers and instruction pointer. This ensures that the Prewarmer can be rolled back to its idle state prior to the injection attempt, maintaining system availability.

V. PERFORMANCE PREDICTOR

To accurately estimate performance degradation under co-location and ensure strict SLO satisfaction, we design a Performance Predictor. By analyzing job resource characteristics and the intensity of resource contention in an NVIDIA MPS-enabled environment, the Predictor directly estimates the execution latency under interference.

We choose Random Forest as the Predictor because performance degradation under GPU co-location is jointly affected by multiple factors, such as competition for compute resources and contention for memory bandwidth. As a result, the relationship between resource contention and execution latency is typically highly non-linear, making it difficult for simple linear models to characterize accurately. Random Forest can effectively fit such complex mappings through an ensemble of decision trees, and is therefore well suited for our performance prediction task.

The input features are summarized in Table II. We incorporate job features, interference features, and MPS configurations as inputs to predict the execution time of a job under co-location. Job features include SM utilization, memory bandwidth, and L2 cache occupancy during standalone execution. Interference features characterize the resource contention for compute and memory bandwidth from co-resident jobs on the same GPU. The MPS configuration represents the resource percentage allocated to the job. By modeling the interplay between job resource utilization and the execution environment, the Predictor estimates co-located execution time to proactively prevent SLO violations

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

caused by resource contention. Furthermore, we guarantee SLO compliance by validating the predicted execution time against the job's SLO.

Specifically, we implement the Predictor as a Random Forest regressor with 100 regression trees. Each tree is trained on a bootstrap-sampled subset of the original training data, and the maximum depth of each tree is set to 12. During inference, the model averages the outputs of all trees as the final prediction, thereby reducing variance and improving prediction stability and generalization ability. In our implementation, we empirically measure that a single Predictor inference takes approximately 0.5 ms, indicating that the Predictor introduces only negligible runtime overhead.

We construct a dataset comprising 1,000 samples using the tasks listed in Table I. These foundational tasks encompass 16 native CUDA workloads and 32 deep learning workloads. Specifically, we generate the samples by randomly co-locating 2 to 10 concurrent tasks and sweeping the MPS proportional limits from 10% to 100% in 10% increments. Finally, we split the dataset into an 80% training set and a 20% testing set. During training, we use Mean Absolute Error (MAE) as the splitting criterion for tree construction to improve robustness against abnormal interference, and we ultimately adopt Mean Absolute Percentage Error (MAPE) as the primary metric to evaluate the model's generalization and prediction accuracy. Based on this metric, our model achieves an overall prediction error of 3.7%. For different GPU types, we use the same feature format and data collection pipeline, but train an independent Random Forest Predictor for each GPU type. At runtime, FAASHARE selects the corresponding Predictor according to the target GPU type.

VI. SPATIO-TEMPORAL PRIORITY QUEUE SCHEDULING

We design spatio-temporal priority queue scheduling to provide fair execution opportunities for different types of serverless task queues in over-subscribed scenarios, ensuring SLO guarantees. Specifically, we propose a temporal-spatio priority task scheduler to provide fair execution opportunities for scheduling different types of serverless tasks in over-subscribed scenarios, thereby ensuring SLO guarantees.

The input of the algorithm includes the SLO and memory requirements of all tasks in the task queue, as well as the Performance Predictor, and the output is the optimal task execution order. The detailed algorithmic flow is shown in the Algorithm 1. We first sort the oversubscribed job queue according to SLO urgency to prioritize latency-critical jobs (Line 1). Here, decoupling means that execution time is removed from the primary sorting criterion, rather than from the entire scheduling process. After the initial order is fixed, execution-time estimates are only used as a feasibility signal for SLO validation. Then, the job queue is partitioned into several sub-queues based on GPU memory capacity (Line 2). To quickly determine whether concurrent tasks within a sub-queue can collectively meet their respective SLOs under MPS co-location, we tentatively assign each task the minimum MPS configuration that just satisfies its own SLO and invoke the Performance Predictor, where the `MPS_context` consists of contention features and the MPS configuration. If interference still causes any task to violate its SLO, we continuously migrate the task at the tail of the sub-queue to another GPU until the prediction shows that all remaining tasks can safely meet their SLOs (Lines 3-6). We perform space optimization to further maximize the number of concurrently executable tasks within a fixed GPU memory size, thereby reducing the average execution time of the entire queue. Specifically, we sequentially select tasks that satisfy the constraints from different queues for order swapping until we have traversed the entire queue (Lines 7-9). The purpose of swapping task orders is to use the same amount of memory

Algorithm 1: Spatio-Temporal Priority Queue Scheduling Algorithm With Performance Predictor.

Input: Task queue A with attributes mem, $time$, and SLO, Performance Predictor
Output: Optimized task queue A
1: `SortBySLO` A;
2: $SubQueues \leftarrow$ `PartitionByMemory` A, M_{GPU};
3: **foreach** Q in $SubQueues$ **do**
4: **Assign minimal MPS to each** $t \in Q$ **satisfying** $t.SLO$;
5: **while** $\exists t \in Q, PerformancePredictor(t, MPS_Context) > t.SLO$ **do**
6: Transfer $Q.tail()$ to another gpu;
7: **end**
8: **end**
9: **for** $i \leftarrow |SubQueues| - 2$ **to** 0 **do**
10: **for** $j \leftarrow i + 1$ **to** $|SubQueues| - 1$ **do**
11: `ExecuteExchange` $SubQueues[i]$, $SubQueues[j]$
12: **end**
13: **end**
14: **Function**: `ExecuteExchange`Q_1, Q_2 $G_1 \leftarrow \emptyset$, $G_2 \leftarrow \emptyset$;
15: **foreach** t_1 in Q_1 **do**
16: **foreach** t_2 in Q_2 **do**
17: **if** Match(t_1, t_2) **then**
18: $G_1 \leftarrow G_1 \cup \{t_1\}$;
19: $G_2 \leftarrow G_2 \cup \{t_2\}$;
20: **end**
21: **end**
22: **end**
23: **if** $|mem(G_1) - mem(G_2)| \leq \epsilon$ **then**
24: **foreach** t in $G_1 \cup G_2$ **do**
25: $t_{exec} \leftarrow PerformancePredictor(t, MPS_Context)$;
26: **if** $t_{exec} > t.SLO$ **then**
27: **foreach** gpu in $GPUs$ **do**
28: **if** $gpu \neq$ CurrentGPU(t) **and**$mem(t) \leq M_{gpu}$ **then**
29: Transfer t to gpu;
30: **return**;
31: **end**
32: **end**
33: **end**
34: **end**
35: `SwapGroups` G_1, G_2;
36: **if** Avg$(t_{Q_1}, t_{Q_2}) <$ before **then**
37: Apply(G_1, G_2);
38: **end**
39: **return**;

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

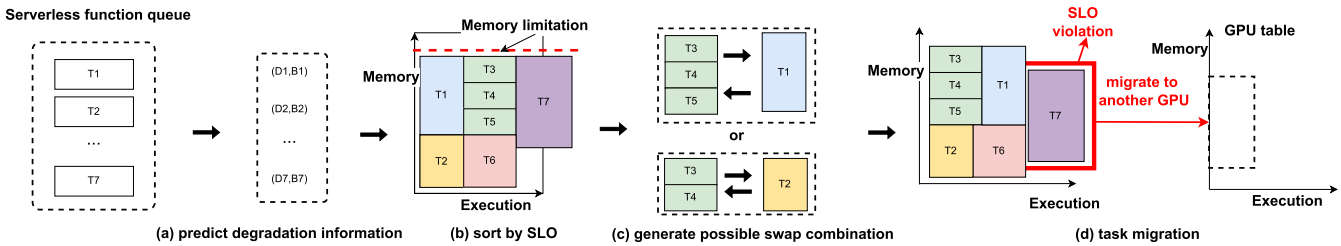


Fig. 7. Spatio-temporal priority queue scheduling.

to substitute in more tasks that can be executed in parallel while satisfying the constraints, thereby achieving task priority updates and reducing the average execution time of the task queue. The task order swapping considers SLO guarantees and average task execution as constraints (Lines 13-18). To further address potential SLO violations caused by interference arising from consolidating multiple workloads on an MPS-enabled GPU, we similarly utilize the Performance Predictor during the execution exchange phase. For each task involved in a potential swap, the Predictor evaluates its actual execution time within the current MPS co-location context. If the predicted execution time exceeds the task's SLO requirement, indicating an SLO violation due to resource contention, the Scheduler intervenes by migrating the affected task to another available GPU to prevent severe interference (Lines 23-29). If a task swap can improve the average task completion time while meeting SLO requirements, pre-verified by the Predictor, we perform the swap to maximize memory utilization and reduce the average completion time (Lines 31-34). In this stage, the Scheduler determines which tasks should run first and which tasks can be grouped together, while the detailed resource allocation is handled in a subsequent optimization stage. Fig. 7 also illustrates the process of spatio-temporal priority queue scheduling. We first generate possible task swap combinations, and then, with the assistance of the Performance Predictor, determine the optimal task execution order by comparing the performance under different task combinations. Additionally, we migrate tasks in the queue that violate their SLOs to a new GPU.

VII. Bayesian Optimization for GPU Sharing

Co-location inevitably introduces resource contention and job performance degradation. Effective resource allocation is the key to improving co-location performance. Bayesian Optimization is an efficient global optimization algorithm that performs a global search by constructing a probabilistic model of the objective function. In our system, BO functions exclusively as a resource-level decision module. Once the upstream Scheduler determines the execution order and candidate co-location set, BO takes over to finely tune the MPS quota for each task within this set. It does not alter task scheduling, but rather determines the optimal GPU share each selected task should receive to maximize overall GPU utilization while strictly satisfying SLO constraints

Thus, we propose a GPU sharing strategy that integrates BO with MPS. Specifically, BO takes jobs and their features as input to generate candidate MPS configurations. Then, the system executes jobs under these candidate configurations and selects the optimal MPS configuration based on the observed data. Finally, BO learns the functional mapping between jobs and MPS configurations to output the optimal configuration for any given set of jobs. In the following sections, we first describe the problem model and then detail the BO-based optimization strategy for serverless computing.

A. Problem Model

Specifically, the input to the optimization model is a 7-dimensional feature vector, and the output is the optimal MPS configuration for a job. Consider a queue containing N concurrent jobs. For any job i, the input features consist of three components: job features v_i, contention features c_i (detailed in Table II), and a candidate MPS configuration s_i for job i. Here, $\mathcal{S}$ denotes the feasible search space of candidate MPS configurations, and $s_i \in \mathcal{S}$ represents the configuration selected for job i. We define the actual execution time of a job under configuration s_i as a function f_i of the input features and the configuration, i.e., $f_i(v_i, c_i, s_i) \rightarrow t_i$. The objective of BO is to explore the objective function space to identify the optimal configuration s_i for each job in the queue, thereby minimizing the job execution time. Meanwhile, the execution time of all jobs must satisfy their respective SLOs. The optimization problem for finding the optimal MPS configuration can be formulated as follows:

$$s_i^* = \arg\min_{s_i \in \mathcal{S}} f_i(v_i, c_i, s_i) \tag{1}$$

$$\text{s.t.} \quad f_i(v_i, c_i, s_i) \leq SLO_i, \quad \forall i \in \{1, \ldots, N\} \tag{2}$$

B. Bayesian Optimization

To adapt BO for the serverless environment, we incorporate several key design enhancements: offline and online optimization, adaptive exploration-exploitation, and a safety guardrail mechanism.

1) Offline and Online Optimization: We decouple the BO process into offline and online phases. In the offline phase, we generate job queues based on the workloads listed in Table I and profile 1000 data points in a real GPU environment, capturing job features, MPS configurations, and the corresponding execution times. We employ a Gaussian Process (GP) with a Matern kernel as the surrogate model to fit high-dimensional non-linear relationships. To evaluate candidate configurations, we utilize

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

Expected Improvement (EI) as the acquisition function. The surrogate model is pre-trained on offline data, and the inference results are cached in an SM configuration table. For different GPU types, FAASHARE adopts the same BO workflow, but maintains independent profiling data, GP surrogate models, and MPS configuration tables for each GPU type. During online serving, FAASHARE first queries the SM configuration table using the job features, contention features, and target GPU type. If a matching entry is found, the corresponding MPS configuration is directly returned, with a lookup overhead of about 0.2 ms. Otherwise, FAASHARE selects the closest historical configuration in the normalized feature space and immediately applies it to the job. The runtime measurements are recorded and used to trigger asynchronous BO updates on idle GPUs. The newly explored configurations are then inserted into the SM configuration table, enabling future similar requests to reuse them directly. Therefore, the online serving path only involves table lookup and historical configuration retrieval, while BO exploration is performed asynchronously.

2) Optimization of Exploration and Exploitation: We use the GP as a surrogate model for Bayesian optimization, which efficiently deals with complex nonlinear relationships and provides precise uncertainty estimates. We use EI as the acquisition function because it provides an intuitive and effective way to balance the needs of exploration and utilization, which allows the optimization process to use existing information to find regions of performance improvement and explore regions with higher uncertainty to discover potentially better solutions.

To make EI adapt to the optimization of SM configuration for complex tasks in serverless, we design an adaptation optimization for BO, which dynamically identifies the extent of exploration and exploitation in the EI converge process. Specifically, we dynamically adjust the ratio of exploration to utilization by iteratively setting exploration parameters to quickly find the optimal configuration to reduce the number of iterations. We can calculate as follows.

$$\mathrm{EI}(x) = \left(\mu(x) - f\left(x^{+}\right) - \xi\right)\Phi(Z) + \sigma(x)\phi(Z) \tag{3}$$

$$Z = \frac{\mu(x) - f\left(x^{+}\right) - \xi}{\sigma(x)} \tag{4}$$

where $\mu(x)$ is the mean of the prediction at the point x, $\sigma(x)$ denotes the predicted standard deviation at the point x, $f(x^{+})$ is currently the best known observation, ξ is the ratio of exploration and exploitation used to balance the exploration and exploitation weights. Besides, ξ directly affects Z and in turn affects the size of EI. EI tends to improve exploration when ξ is large, and when ξ is small, it tends to develop known good regions, which can adjust the balance between exploration and exploitation at different stages of optimization by dynamically adjusting the values of ξ.

To prevent BO from generating invalid MPS configurations, we introduce a safety guardrail mechanism based on the lower bound of SLOs. Upon detecting an invalid configuration, the system intercepts the request and bypasses the surrogate model, directly assigning the minimum MPS resources required to satisfy the job's SLO. This mechanism ensures that SLOs remain inviolate even if the BO model fails in extreme cases.

VIII. IMPLEMENTATION

The prototype is implemented in approximately 5 K lines of code, with Python as the primary control language, combined with CUDA/MPS for fine-grained device-side resource control, and lightweight RPC mechanisms for state synchronization and command dispatch between the control plane and node agents. In the current prototype implementation, an MSContainer runs as a privileged container to allow the Proxy to use `ptrace`. We do not configure additional Linux capabilities separately. To limit the trusted boundary, the Prewarmer installs a `seccomp` filter before executing user code, which blocks unsafe kernel-level operations, such as `ptrace`. An MSContainer can contain multiple Proxy–Prewarmer pairs, which share the same PID namespace inside the MSContainer. The MSContainer maintains the mapping between Proxy and Prewarmer PIDs to ensure that each Proxy only attaches to its corresponding Prewarmer. Therefore, FAASHARE does not require `hostPID` or cross-container PID visibility in the Kubernetes deployment.

To extend the system from a single GPU to multiple GPUs, we mainly make three modifications. First, we extend the original local GPU status table into a cluster-wide resource table that uniformly records the load, memory usage, and MPS quotas of GPUs across all nodes. Second, we extend the existing scheduling flow into a two-level scheduling process: the system first selects the target node and target GPU, and then reuses the existing queue scheduling and BO optimization logic within the selected GPU. Third, we maintain an independent local MPS control instance for each GPU, while node agents continuously report device status and receive configuration commands.

Regarding the adaptation to Multi-Instance GPU (MIG)[4], the current design of FAASHARE mainly focuses on MPS-based GPU sharing and cannot yet use BO to simultaneously determine the MPS quota and MIG partition for a task or a group of tasks. Nevertheless, FAASHARE can still be adapted to MIG-capable GPUs. Since MIG can partition a physical GPU into multiple hardware-isolated GPU instances, each providing relatively independent compute, memory, and cache resources, FAASHARE can treat each MIG instance as an independent scheduling unit. In such a deployment, the system needs to additionally design a MIG allocation method to first assign multiple tasks to different GPU instances. Then, MPS can be enabled within each MIG instance, and the BO-based MPS quota optimization logic can be applied inside each MIG instance to support both MIG- and MPS-level optimization. We leave the joint optimization of MIG partitioning and MPS quota configuration as future work.

IX. PERFORMANCE EVALUATION

A. Experimental Environment and Setup

1) Testbed: We use two servers equipped with GPUs for our evaluations. Specifically, one server is configured with 8

[4]https://docs.nvidia.com/datacenter/tesla/mig-user-guide/index.html

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

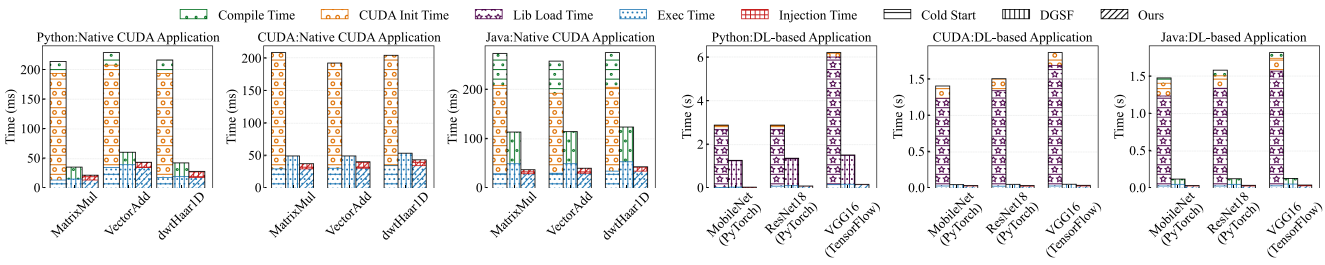


Fig. 8. Breakdown of each step of our workloads when running with unoptimized and dynamic injection.

NVIDIA RTX 3090 GPUs (24 GB memory per GPU), and the other server is configured with 8 NVIDIA RTX 4090 GPUs (48 GB memory per GPU), for a total of 16 GPUs across both servers.

2) Workloads: We follow the workload settings in MISO [38] and synthesize a job trace inspired by Helios [39]. In our testbed experiments, we instantiate 120 jobs in total, including 60 SLO-constrained (latency-critical) jobs and 60 best-effort jobs, uniformly sampled from Table I. For best-effort jobs, we treat DL workloads as model training tasks, while for native workloads we use those marked with "-" in Table I (i.e., without SLO). Job arrivals are generated by a Poisson process with an average inter-arrival time of $\lambda = 4$ seconds. To evaluate SLO compliance, we follow [40] to set the SLO for each task to 4 times its standalone execution time measured on NVIDIA RTX 4090 GPUs, and the resulting SLO values are reported in Table I. Following the methodology in DVABatch [41], we define low, medium, and high loads as 25%, 60%, and 90% of the peak throughput, respectively. These load levels are determined by subjecting each benchmark to a stepping load to identify its maximum processing capacity.

3) Metrics: We evaluate the system performance using three key metrics [38]: Average Job Completion Time (JCT), System Throughput (STP), and Makespan. Average JCT represents the duration from the beginning of a request's execution to its completion, which serves as an indicator of the system's parallel execution optimization. STP measures the number of tasks completed or the amount of data processed per unit of time, reflecting the overall performance efficiency. Finally, Makespan refers to the completion time of the longest-running job, characterizing the total time required to finish the entire workload.

4) Baselines: **TGS** [42] achieves task sharing by dynamically adjusting the kernel priority of tasks. **DGSF** [43] transparently enables serverless functions to use GPUs through general-purpose APIs. **ElasticFlow** [24] is an elastic serverless training platform for distributed deep learning that provides performance guarantees by elastically scaling resources to meet job deadlines. **FaST-GShare** [25] is an efficient FaaS-oriented spatio-temporal GPU sharing architecture that utilizes a dedicated manager to isolate resources and a profiler to ensure SLO compliance for deep learning inference. **FluidFaaS** [44] provides a dynamic pipelined solution for serverless computing that achieves strong isolation and improved throughput through fine-grained GPU sharing. **Priority-time queue scheduling (PT)** schedules serverless tasks based on execution time priority, assigning higher priority to tasks with shorter execution times. **Priority-memory queue scheduling (PM)** schedules serverless tasks based on memory footprint, where tasks with a smaller memory footprint have higher priority.

B. Cold Start Optimization

1) Performance Analysis of Cold Start: We evaluate the performance of dynamic injection under different workloads as shown in Fig. 8. For workloads implemented in C language, the composition of cold start time primarily includes three key components: CUDA initialization, execution time, and injection time.

FaaShare reduces the latency by 10 × compared to the baseline, as FaaShare can preload and initialize the necessary CUDA environments before program execution to eliminate CUDA initialization time. FaaShare also achieves a 5% performance improvement over DGSF because FaaShare does not need to consider synchronization operations of operator execution. For native CUDA applications with typically short execution times (e.g., MatrixMul) and low-load model inference tasks (e.g., ResNet-18), FaaShare significantly reduces the overall cold start latency. Additionally, the injection time for FaaShare is 12 ms.

2) Multi-programming Language Extension: For the programming languages Python and Java, we breakdown the execution times of workloads into compilation time, CUDA initialization, library initialization, execution time, and injection time. For native CUDA applications, FaaShare can further reduce latency compared to baseline and DGSF, achieving performance improvements of 5x and 10x respectively. Because FaaShare not only eliminates compilation time (20 ms) and CUDA initialization but also optimizes library latency.

For ML applications, FaaShare offers even more significant performance enhancements. ResNet-18 relies not only on PyTorch but also on cuDNN, which introduces delays of approximately 0.6 s and 1.2 s respectively. FaaShare can completely remove the loading overhead of these libraries and improve performance by 10x and 10x, achieving low latency. Notably, the time cost of dynamic injection based on Python and Java ranges from 5-10 milliseconds.

3) Sensitivity Analysis of Checkpoint for Cold Start: When errors occur in the program, causing it to terminate abruptly, as mentioned in IV, the termination of the program also leads to the termination of the pre-warming process, which requires the pre-warming process to be restarted to load the necessary environment in advance. To demonstrate the effectiveness of

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

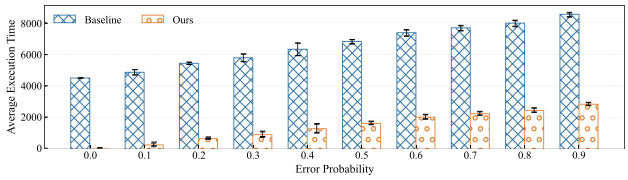


Fig. 9. Error sensitivity analysis for cold start.

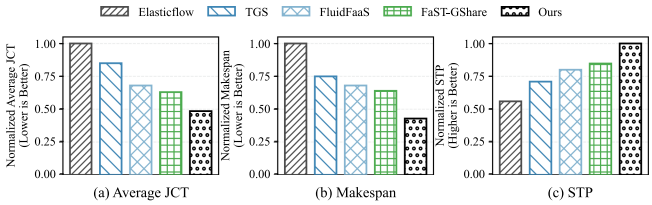


Fig. 10. Performance comparison between different strategies of GPU sharing.

checkpoint for cold start, we set the same error probability for 10 tasks and evaluate the average execution time of tasks under different error probabilities, with experimental results shown in Fig. 9. When the program error probability is 0.5, FAASHARE achieves 5× improvement compared to baseline because only a few programs need to be restarted. As the error probability increases, both methods incur higher execution time due to more frequent failures and re-executions, while FAASHARE still avoids complete Prewarmer restart and dependency reloading. When the error probability is 0.9, FAASHARE reduces the latency by 2.3× compared to baseline. The primary time cost is spent on continuously compiling and loading dependency packages. At the same time, FAASHARE can restore the context through `ptrace` and re-run the modified files by the user without needing a complete restart.

C. Overall Performance Comparison

1) Overall Performance Comparison: FAASHARE significantly improves GPU co-location performance through MSContainer, spatio-temporal scheduling, and Bayesian Optimization (BO). Fig. 10 shows the performance of the different methods in terms of average JCT, makespan, and STP. The overall performance, from lowest to highest, is: ElasticFlow, TGS, FluidFaaS, FaST-GShare, and FAASHARE.

First, as shown in Fig. 10(a), FAASHARE reduces the average JCT by 51%, 43%, 29%, and 23% compared to ElasticFlow, TGS, FluidFaaS, and FaST-GShare, respectively. ElasticFlow and TGS perform the poorest due to their lack of spatial sharing, but TGS slightly outperforms ElasticFlow by leveraging operator rate control. FluidFaaS and FaST-GShare improve performance by introducing spatial isolation, with the MPS-based FaST-GShare outperforming the MIG-only FluidFaaS. In contrast, FAASHARE achieves the best results by minimizing environment initialization latency through MSContainer.

In Fig. 10(b), FAASHARE reduces the makespan by 57%, 43%, 37%, and 33% compared to the aforementioned methods, respectively. The concurrency efficiency of spatial sharing methods is significantly better than that of time-sharing methods, and

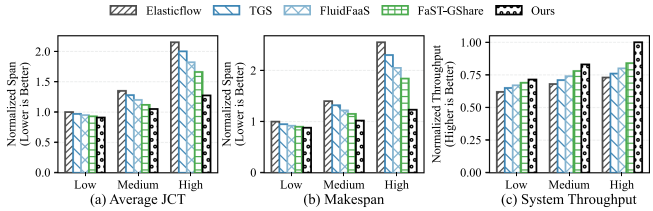


Fig. 11. Multi-scenario performance evaluation.

FaST-GShare surpasses FluidFaaS owing to the throughput advantage of MPS. FAASHARE further maximizes the concurrency opportunities for all jobs through spatio-temporal scheduling and utilizes BO to allocate computing resources reasonably and avoid outliers, thereby taking a substantial lead.

In Fig. 10(c), FAASHARE achieves the highest STP, improving upon ElasticFlow, TGS, FluidFaaS, and FaST-GShare by 79%, 41%, 25%, and 18%, respectively. By configuring the most cost-effective resources for different workloads using BO, FAASHARE maximizes GPU resource utilization and efficiently achieves task parallelism, enabling each job to attain high execution efficiency.

2) Performance Comparison Across Multi-Load: We compare FAASHARE against ElasticFlow, TGS, FluidFaaS, and FaST-GShare under low, medium, and high load scenarios. As shown in Fig. 11, FAASHARE consistently outperforms the baselines across all scenarios. Under medium load, the system throughput of FAASHARE reaches 1.19× that of ElasticFlow. Under high load, FAASHARE reduces the average JCT by 40%, 36%, 30%, and 23% compared to the four methods, respectively. This improvement is attributed to its superior spatial isolation over traditional time-sharing, as well as the reduction in initialization latency by MSContainer. Overall, FAASHARE achieves significant performance improvements across all load scenarios, and notably optimizes execution efficiency, especially in highly competitive environments.

3) Exploration and Exploitation Sensitivity Study: The exploration degree of the acquisition function in BO determines the confidence in finding the optimal value in the target region, primarily reflected in the variable ξ.

Fig. 12 presents a sensitivity analysis of different values in the convergence process of BO under different workload scenarios. First, in the low workload scenario, ξ values of 0.01 and 0.05 exhibit better performance and smaller bias compared to ξ values of 0.1 and 0.2. Moreover, smaller ξ values enable BO to converge faster in finding the optimal point. Because when ξ is small, the acquisition function tends to further search for the optimal point in the known better region, reducing the search time. On the other hand, when ξ is large, the acquisition function tends to explore unknown regions, which results in more time wasted on exploration.

In the medium and high workload scenarios, smaller ξ values still demonstrate better performance and smaller bias. Additionally, the convergence process generally exhibits a smaller bias compared to the low workload scenario, and the impact of different ξ values on the convergence process gradually diminishes. This is attributed to the gradual reduction in the number of tasks

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

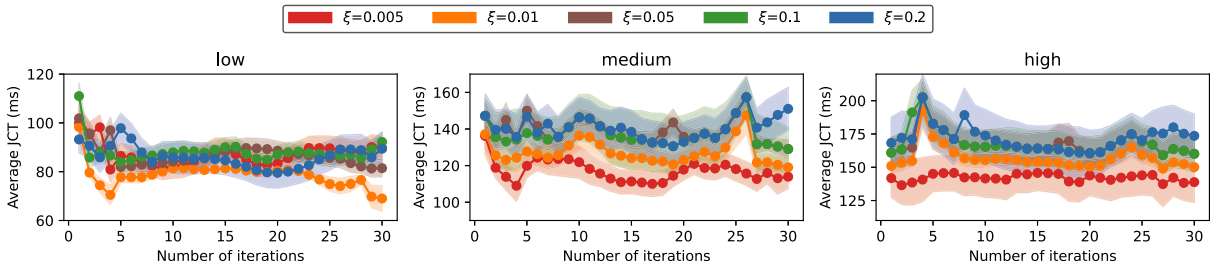


Fig. 12. Exploration and exploitation evaluation.

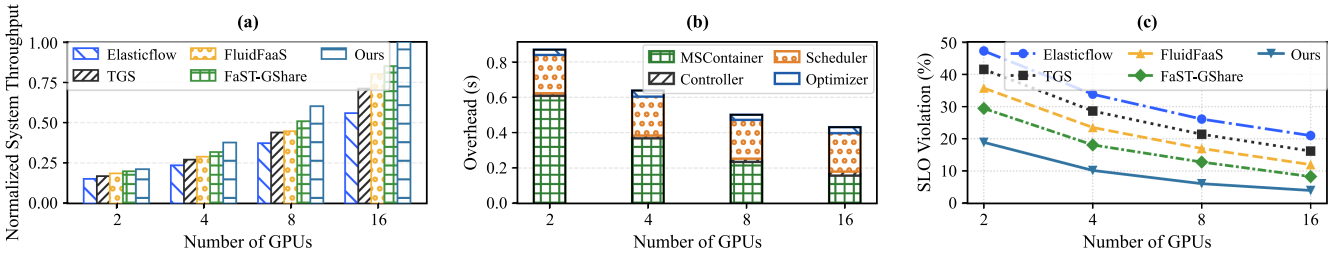


Fig. 13. Performance comparison and overall component-wise overhead in multi-GPU environments. (a) normalized throughput comparison; (b) breakdown of component overheads; (c) comparison of SLO violation rates.

executed in parallel in the medium and high workload scenarios, and the corresponding decrease in the need for BO to optimize SM configurations, resulting in faster convergence.

D. Performance Scaling With Different Numbers of GPUs

To evaluate the performance of our approach in multi-GPU environments, we sequentially report four sets of results: multi-GPU throughput, system overhead breakdown, SLO violation rate, and P99 tail latency. Throughput reflects the scaling trend, overhead illustrates the cost of the control plane and optimization modules, and SLO alongside P99 tail latency characterize the changes in SLO under co-located and scaling conditions.

To evaluate the throughput performance in multi-GPU environments, we measure the normalized system throughput across different cluster sizes. As shown in As shown in Fig. 13(a), the throughput of all methods increases with the number of GPUs. Our method exhibits the best scalability, improving throughput by 79%, 41%, 25%, and 18% compared to ElasticFlow, TGS, FluidFaaS, and FaST-GShare, respectively under the 16-GPU scale. ElasticFlow suffers from coarse-grained allocation, TGS lacks spatial sharing, FluidFaaS misses multi-GPU coordination and has limited parallelism, and FaST-GShare experiences a certain degree of computing resource idleness. In contrast, our framework dynamically searches for the optimal MPS configuration via BO and leverages a spatiotemporal priority mechanism to achieve highly efficient parallelism for heterogeneous workloads, thereby substantially increasing the overall throughput.

Fig. 13(b) illustrates the execution overheads of the four core components of our system across different cluster sizes, including MSContainer, Controller, Scheduler, and Optimizer. We report the total overhead of each component throughout the entire experimental workflow, rather than the per-task component overhead. As the number of GPUs increases from 2 to 16, the overhead of MSContainer, which is responsible for function environment warm-up and dynamic injection, steadily decreases from 0.608 s to 0.157 s. This is primarily attributed to the accelerated parallel initialization brought by the larger cluster scale. Meanwhile, the Controller, which is responsible for global state maintenance, introduces only 0.015 s to 0.023 s of overhead across different GPU scales. The Scheduler, which performs spatio-temporal priority scheduling and predictor-based feasibility checks, remains almost stable, with an overhead of about 0.217 s to 0.220 s. For BO, since the overhead mainly comes from the SM configuration table lookup, and the configuration query for each task is independent of scenario changes, its overhead remains within a relatively stable range of approximately 0.02 s to 0.03 s across different scenarios. Overall, each component introduces only limited control-plane overhead, demonstrating that FaaShare has good scalability in multi-GPU scenarios.

To evaluate the SLO guarantee capabilities in multi-GPU environments, we test the SLO violation rate across different cluster sizes. As shown in Fig. 13(c), as computing capacity expands, the SLO violation rates of all methods exhibit a downward trend. Although all methods face compute bottlenecks in the 2-GPU scenario, our method rapidly converges the violation rate under 4-to-16 GPU configurations, further suppressing it from 10.2% to an extremely low level of 4.0%.

Furthermore, we evaluate the P99 tail latency across 10 different workloads in Fig. 14. Overall, ElasticFlow and TGS are more prone to queueing tails due to their limited resource allocation and sharing capabilities. FluidFaaS and FaST-GShare lack dynamic coordination and adaptive adjustment during multi-GPU scaling, resulting in more pronounced tail jitter in scenarios involving co-location contention and fragmentation.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

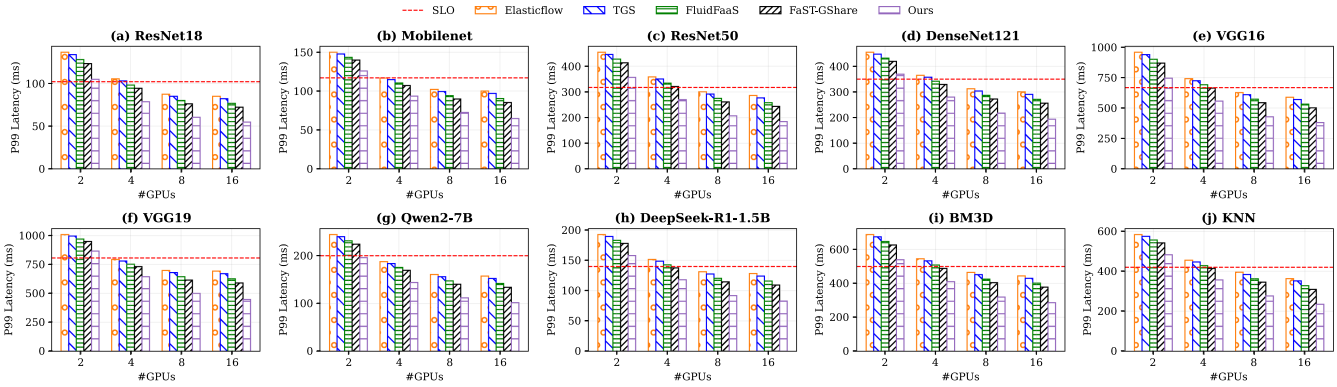


Fig. 14. Performance comparison of P99 latency.

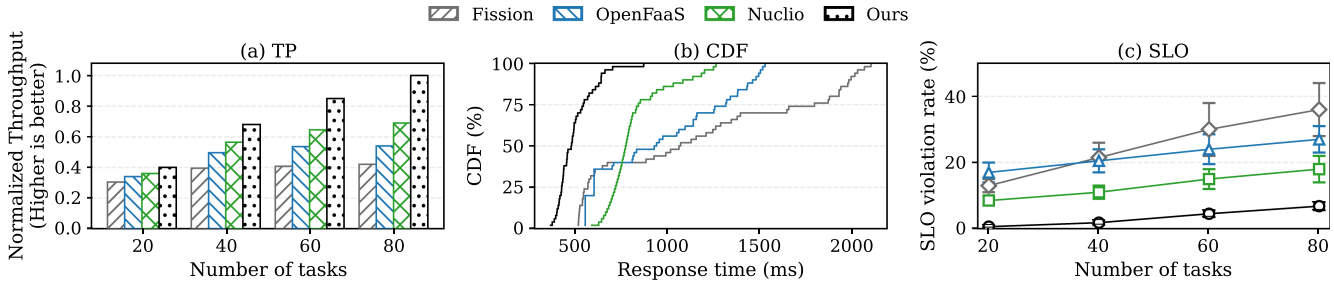


Fig. 15. Performance comparison of various open-source serverless frameworks. (a) Normalized throughput across different numbers of tasks; (b) Cumulative Distribution Function (CDF) of response latency; (c) Comparison of SLO violation rates under different numbers of tasks.

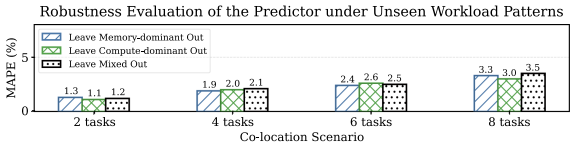


Fig. 16. Robustness evaluation of the Performance Predictor under unseen workload patterns.

Our approach combines spatiotemporal priority scheduling with Bayesian-optimized SM quota selection to mitigate co-location interference and stabilize tail latency. The performance gap between the methods widens as the cluster size expands. Under the 16-GPU configuration, our method achieves lower P99 tail latency on most workloads, and this advantage becomes more prominent with increased cluster scales.

E. Comparison With Open-Source FaaS Frameworks

We evaluate the performance of the platforms in scaling tests ranging from 20 to 80 concurrent tasks. As shown in Fig. 16(a) and 16(b), when the number of concurrent tasks reaches 80, the throughput of FAASHARE improves upon Fission, OpenFaaS, and Nuclio by approximately 133%, 85%, and 45%, respectively, while its SLO violation rate remains at 6.8%. In contrast, the SLO violation rates of Fission and OpenFaaS increase to 36.0% and 27.0%, respectively, and to 18.0% for Nuclio. The cumulative distribution function (CDF) of response time in Fig. 16(c) further corroborates this trend; the 99th percentile latency of FAASHARE is reduced by 62%, 48%, and 37% compared to the three aforementioned frameworks, respectively. The performance of traditional frameworks is primarily constrained by their general-purpose architectural designs. Fission's frequent routing allocation and OpenFaaS's dual-layer gateway queuing mechanism introduce additional network hops and lead to long-tail latency. Although Nuclio eliminates external communication overhead by leveraging an in-pod multi-worker concurrent model, it lacks GPU-specific optimization. To address this, FAASHARE uses MSContainer dynamic injection to eliminate the cold start bottleneck, incorporates spatio-temporal priority queue scheduling to guarantee the SLO of co-located tasks, and employs Bayesian optimization combined with MPS to allocate SM resources, which mitigates resource contention, compresses queuing latency, and improves overall system throughput and GPU utilization.

F. Robustness Evaluation of the Performance Predictor

To evaluate the robustness of the predictor under various workload combinations, we conduct supplemental generalization experiments. We first construct co-located queues for four scenarios with 2, 4, 6, and 8 jobs, respectively. Based on the ratio of memory-intensive tasks (R_{mem}), we categorize these workloads into three patterns: Memory-dominant ($R_{mem} > 50\%$), Compute-dominant ($R_{mem} < 50\%$), and Mixed ($R_{mem} = 50\%$). We sample 300 instances for each scenario from the dataset in Section V and adopt a leave-one-pattern-out validation strategy, where one pattern is reserved as the test set and the

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

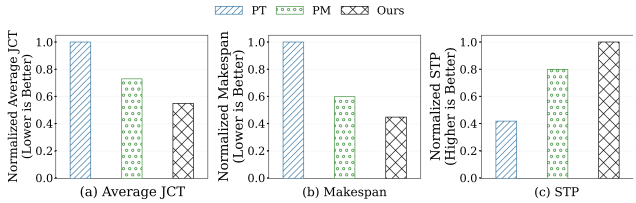


Fig. 17. Performance comparison of priority queue.

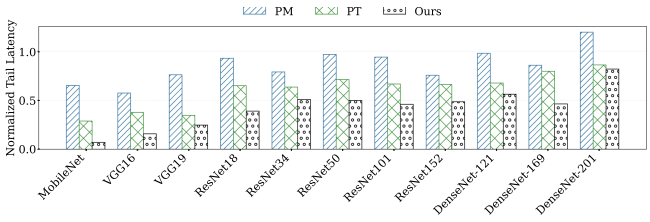


Fig. 18. Normalized P99 tail latency.

remaining two are used for training. Evaluation results show that across the four scenarios, the average MAPE of the Predictor for the three unseen combinations is 1.2%, 2%, 2.5% and 3.2%, respectively. Although the error exhibits an upward trend as the number of jobs increases and combinations become more complex, the Predictor maintains low error performance and demonstrates strong generalization capability.

G. Load-Aware Priority Queue Optimization

1) Tail Latency Analysis of Priority Queue: To investigate the impact of priority strategies on the task queue, we conduct performance analysis experiments, as shown in Fig. 17. In Fig. 17(a), compared to PT and PM, we achieve 41% and 23% improvement in average JCT. This is due to the dynamic task priority adjustment strategy, which allows different tasks to have a fairer chance of execution. PT focuses too much on execution time and overlooks the memory space occupancy, resulting in a limited number of tasks executed in parallel. PM fails to provide fair opportunities for execution time. In Fig. 17 (b), we continue to provide 51% and 19% improvement in Makespan. PT leads to delays in tasks with long execution time but low memory occupancy, while PM leads to delays in tasks with short execution time but high memory occupancy. Ultimately, in Fig. 17 (c), we still achieve 2.5 $\times$ higher and 20% improvement in STP compared to PT and PM.

2) Tail Latency Analysis of Priority Queue: Fig. 18 shows the results of different strategies on p99 latency compared to baseline. First, benefit from the dynamic adjustment of the priority queue, we can significantly reduce the tail latency of application requests. FAASHARE reduces the P99 tail latency by up to 5$\times$ compared with the PM and PT baselines. As the application workload increases, the gains obtained gradually diminish. This is because PT focuses only on the execution time of tasks while ignoring the impact of memory occupancy, resulting in high tail latency for longer-running tasks. Similarly, PM only considers memory occupancy and prioritizes tasks with lower memory usage, but it overlooks the factor of execution time. By dynamically adjusting the execution time and memory occupancy, we achieve a trade-off between execution time and memory occupancy.

X. Related Work

GPU-enabled Serverless Computing: Molecule [45] is the serverless computing system utilizing heterogeneous computers and enabling both general-purpose devices and domain-specific accelerators for serverless applications. Kim et al. develop an API that connects the open source framework to the NVIDIA-Docker and commands that enable GPU programming, then proposes a GPU-supported serverless computing framework that can deploy services faster [46]. Satzke et al. propose to extend the open-sourced KNIX high-performance serverless framework to enable executing functions on shared GPU cluster resources [47]. DGSF [43] transparently enables serverless functions to use GPUs through general-purpose APIs. Kernel-as-a-service (KaaS) [48] implements a GPU interface for stateless functions to access GPU instances.

Cold Start Optimization: Shahrad et al. [49] manage a pre-warming window and a keep-alive window for each stateless function to alleviate performance degradation caused by cold starts. INFless [16] proposes long-short term histogram, which simultaneously tracks and calculates the histograms of idle time for long-term and short-term applications to find the appropriate warm-up window and duration, but it reduces the cold start of functions at the container level.

Agarwal et al. [50] propose a reinforcement learning agent setting to analyze the identified factors such as function CPU utilization, ascertain the function-invocation patterns, and reduce the function cold start frequency by preparing the function instances in advance. FaasCache [51] realizes a caching-inspired greedy-dual keep-alive policy to reduce the cold-start overhead.

GPU Sharing: Prior works have tried to support GPU sharing at different levels. Gandiva [52] proposes GPU time-sharing in shared GPU clusters through checkpointing at low GPU memory usage of the training job. Harmony [53] applies an RL model to make placement decisions to minimize interference and maximize the throughput for bin-packing DL workloads in a GPU data center. Salus [54] focuses on fine-grained GPU sharing with two primitives: fast job switching and memory sharing. Ali-MLaaS [55] comprehensively analyzes large-scale workload traces in Alibaba and discloses the benefit of GPU sharing in production GPU data centers. FaST-GShare [25] considers task and resource allocation under non-computational resource contention, which is orthogonal to our works.

XI. Conclusion and Future Work

In this paper, we present FaaShare, a low-latency, satisfaction of SLO, high GPU utilization serverless framework. For low latency, we design MSContainer to provide pre-warm context to reduce cold start for serverless function. For the satisfaction of SLO, we propose a temporal-spatio priority queue scheduling mechanism. For GPU sharing, we propose a BO-based resource configuration strategy, which significantly improves the performance of the system. The proposed framework proves its

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

effectiveness in extensive experiments. Compared with state-of-the-art baselines, FAASHARE significantly reduces cold-start delay and further improves the average JCT, makespan, and STP by 23%, 33%, and 18%, respectively, over the strongest baseline in GPU-sharing performance.

In future work, we plan to design a dedicated serverless framework for large language model workloads, considering their unique requirements in model loading, KV-cache management, and long-running inference. We also plan to further explore the integration of MIG and MPS to enhance GPU sharing capability while providing stronger resource isolation for serverless functions.

REFERENCES

[1] E. Jonas et al., "Cloud programming simplified: A Berkeley View on serverless computing," 2019, *arXiv:1902.03383*.

[2] Z. Li, L. Guo, J. Cheng, Q. Chen, B. He, and M. Guo, "The serverless computing survey: A technical primer for design architecture," *ACM Comput. Surv.*, vol. 54, pp. 1–34, 2021.

[3] H. Shafiei, A. Khonsari, and P. Mousavi, "Serverless computing: A survey of opportunities, challenges, and applications," *ACM Comput. Surv.*, vol. 54, pp. 1–32, 2019.

[4] L. Zhao, Y. Yang, Y. Li, X. Zhou, and K. Li, "Understanding, predicting and scheduling serverless workloads under partial interference," in *SC21: Int. Conf. High Perform. Comput. Netw. Storage Anal.*, pp. 1–14, 2021.

[5] L. Jin et al., "Ephemera: Accelerating I/O-intensive serverless workloads with a harvested in-memory file system," *ACM Trans. Archit. Code Optim.*, vol. 22, no. 3, pp. 1–24, 2025.

[6] J. Liu et al., "SMore: Enhancing GPU utilization in deep learning clusters by serverless-based co-location scheduling," in *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 5, pp. 903–917, May 2025.

[7] Z. Cai, Z. Chen, R. Ma, and H. Guan, "SMSS: Stateful model serving in metaverse with serverless computing and GPU sharing," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 3, pp. 799–811, Mar. 2024.

[8] R. Moreno-Vozmediano, R. S. Montero, and I. M. Llorente, "IaaS cloud architecture: From virtualized datacenters to federated cloud infrastructures," *Computer*, vol. 45, no. 12, pp. 65–72, Dec. 2012.

[9] E. V. Eyk, L. Toader, S. Talluri, L. Versluis, A. Uta, and A. Iosup, "Serverless is more: From PaaS to present cloud computing," *IEEE Internet Comput.*, vol. 22, no. 5, pp. 8–17, Sep./Oct. 2018.

[10] X. Han et al., "Hermes: Memory-efficient pipeline inference for large models on edge devices," in *Proc. Int. Conf. Comput. Des.*, 2024, pp. 454–461.

[11] H. Wang, D. Niu, and B. Li, "Distributed machine learning with a serverless architecture," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2019, pp. 1288–1296.

[12] J. Jiang et al., "Towards demystifying serverless machine learning training," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 857–871.

[13] J. Carreira, P. Fonseca, A. Tumanov, A. Zhang, and R. Katz, "Cirrus: A serverless framework for end-to-end ML workflows," in *Proc. ACM Symp. Cloud Comput.*, 2019, pp. 13–24.

[14] F. Xu, Y. Qin, L. Chen, Z. Zhou, and F. Liu, "DNN: Achieving predictable distributed DNN training with serverless architectures," *IEEE Trans. Comput.*, vol. 71, no. 2, pp. 450–463, Feb. 2021.

[15] J. Thorpe et al., "Dorylus: Affordable, scalable, and accurate GNN training with distributed CPU servers and serverless threads," in *Proc. USENIX Symp. Operating Syst. Des. Implementations*, 2021, pp. 495–514.

[16] Y. Yang et al., "INFless: A native serverless system for low-latency, high-throughput inference," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2022, pp. 768–781.

[17] J. Li, L. Zhao, Y. Yang, K. Zhan, and K. Li, "Tetris: Memory-efficient serverless inference through tensor sharing," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 473–488.

[18] J. Jarachanthan, L. Chen, F. Xu, and B. Li, "AMPS-INF: Automatic model partitioning for serverless inference with cost efficiency," in *Proc. Int. Conf. Parallel Process.*, 2021, Art. no. 14.

[19] A. Ali, R. Pinciroli, F. Yan, and E. Smirni, "BATCH: Machine learning inference serving on serverless platforms with adaptive batching," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, Art. no. 69.

[20] L. Ao, L. Izhikevich, G. M. Voelker, and G. Porter, "Sprocket: A serverless video processing framework," in *Proc. ACM Symp. Cloud Comput.*, 2018, pp. 263–274.

[21] J. Spillner, C. Mateos, and D. A. Monge, "FaaSter, better, cheaper: The prospect of serverless scientific computing and HPC," in *Proc. 4th Latin Amer. Conf. High Perform. Comput.*, 2017, Buenos Aires, 2018, pp. 154–168.

[22] H. Fingler, Z. Zhu, E. Yoon, Z. Jia, E. Witchel, and C. J. Rossbach, "DGSF: Disaggregated GPUs for serverless functions," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 739–750.

[23] M. Yu et al., "Torpor: GPU-enabled serverless computing for low-latency, resource-efficient inference," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC 25)*, 2025, pp. 597–612.

[24] D. Gu et al., "ElasticFlow: An elastic serverless training platform for distributed deep learning," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2023, pp. 266–280.

[25] J. Gu, Y. Zhu, P. Wang, M. Chadha, and M. Gerndt, "FaST-GShare: Enabling efficient spatio-temporal GPU sharing in serverless computing for deep learning inference," in *Proc. 52nd Int. Conf. Parallel Process.*, 2023, pp. 635–644.

[26] S. Nastic et al., "A serverless real-time data analytics platform for edge computing," *IEEE Internet Comput.*, vol. 21, no. 4, pp. 64–71, 2017.

[27] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving heterogeneous machine learning models on multi-GPU servers with spatio-temporal sharing," in *Proc. 2022 USENIX Annu. Tech. Conf.*, 2022.

[28] H. Abdi and L. J. Williams, "Principal component analysis," *Wiley Interdiscipl. Rev.: Comput. Statist.*, vol. 2, no. 4, pp. 433–459, 2010.

[29] S. Zhang, X. Li, M. Zong, X. Zhu, and R. Wang, "Efficient kNN classification with different numbers of nearest neighbors," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 5, pp. 1774–1785, May 2018.

[30] K. Dabov, A. Foi, V. Katkovnik, and K. O. Egiazarian, "Image denoising by sparse 3-D transform-domain collaborative filtering," *IEEE Trans. Image Process.*, vol. 16, no. 8, pp. 2080–2095, Aug. 2007.

[31] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. pattern Recognit.*, 2016, pp. 770–778.

[32] G. Huang, Z. Liu, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. 2017 IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2261–2269.

[33] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[34] M. Sandler, A. G. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.

[35] Q. Team, "Qwen2.5 technical report," 2024, *arXiv:2412.15115*.

[36] A. Grattafiori et al., "The LLaMA 3 herd of models," 2024, *arXiv: 2407.21783*.

[37] D. Guo et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.

[38] B. Li, T. Patel, S. Samsi, V. Gadepally, and D. Tiwari, "MUSO: Exploiting multi-instance GPU capability on multi-tenant GPU clusters," in *Proc. Symp. Cloud Comput.*, 2022, pp. 173–189.

[39] Q. Hu, P. Sun, S. Yan, Y. Wen, and T. Zhang, "Characterization and prediction of deep learning workloads in large-scale GPU datacenters," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, Art. no. 104 .

[40] J. Wang, Y. Wang, M. Han, and R. Chen, "Colocating ML inference and training with fast GPU memory handover," in *Proc. USENIX Annu. Tech. Conf.*, 2025, pp. 1657–1675.

[41] W. Cui et al., "DVABatch : Diversity-aware Multi-Entry Multi-Exit batching for efficient processing of DNN services on GPUs," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 183–198.

[42] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, "Transparent GPU sharing in container clouds for deep learning workloads," in *Proc. 20th USENIX Symp. Networked Syst. Des. Implementation*, 2023, pp. 69–85.

[43] H. Fingler, Z. Zhu, E. Yoon, Z. Jia, E. Witchel, and C. J. Rossbach, "DGSF: Disaggregated GPUs for serverless functions," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 739–750.

[44] X. Hui, Y. Xu, and X. Shen, "FluidFaas: A dynamic pipelined solution for serverless computing with strong isolation-based GPU sharing," in *Proc. 34th Int. Symp. High- Perform. Parallel Distrib. Comput.*, 2025, pp. 1–14.

[45] D. Du, Q. Liu, X. Jiang, Y. Xia, B. Zang, and H. Chen, "Serverless computing on heterogeneous computers," in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2022, pp. 797–813.

[46] J. Kim, T. J. Jun, D. Kang, D. Kim, and D. Kim, "GPU enabled serverless computing framework," in *Proc. 26th Euromicro Int. Conf. Parallel, Distrib. Netw.-Based Process.*, 2018, pp. 533–540.

[47] K. Satzke et al., "Efficient GPU sharing for serverless workflows," in *Proc. Workshop High Perform. Serverless Comput.*, 2020, pp. 17–24.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.

[48] N. Pemberton, A. Zabreyko, Z. Ding, R. Katz, and J. Gonzalez, "Kernel-as-a-service: A serverless interface to GPUs," 2022, *arXiv:2212.08146*.
[49] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 205–218.
[50] S. Agarwal, M. A. Rodriguez, and R. Buyya, "A reinforcement learning approach to reduce serverless function cold start frequency," in *Proc. IEEE/ACM Int. Symp. Cluster Cloud Internet Comput.*, 2021, pp. 797–803.
[51] A. Fuerst and P. Sharma, "FaasCache: Keeping serverless computing alive with greedy-dual caching," in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2021, pp. 386–400.
[52] W. Xiao et al., "Gandiva: Introspective cluster scheduling for deep learning," in *Proc. USENIX Symp. Operating Syst. Des. Implementations*, 2018, pp. 595–610.
[53] Y. Bao, Y. Peng, and C. Wu, "Deep learning-based job placement in distributed machine learning clusters," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2019, pp. 505–513.
[54] P. Yu and M. Chowdhury, "Fine-grained GPU sharing primitives for deep learning applications," in *Proc. Mach. Learn. Syst.*, 2020, pp. 98–111.
[55] Q. Weng et al., "MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *Proc. USENIX Symp. Networked Syst. Des. Implementation*, 2022, pp. 945–960.



Zhuolong Jiang is currently working toward the Doctoral degree in information security with Shanghai Jiao Tong University, Shanghai, China. His research focuses on serverless GPU optimization.



Zinuo Cai received the bachelor's degree in software engineering from Shanghai Jiao Tong University. He is currently working toward the Graduate degree in computer science with Shanghai Jiao Tong University, Shanghai, China. His research interests include resource schedule and system security in cloud computing.



Yumou Liu is currently working toward the Undergraduate degree in computer science with Shanghai Jiao Tong University, Shanghai, China. His research focuses on cloud computing.



Ruhui Ma received the PhD degree in computer science from Shanghai Jiao Tong University. He is currently an associate professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include cloud computing systems, AI systems, and machine learning.



Rajkumar Buyya (Fellow, IEEE) is currently a Redmond Barry distinguished professor and the director with Quantum Cloud Computing and Distributed Systems Laboratory, School of Computing and Information Systems, University of Melbourne, Australia. He has authored more than 625 publications and seven text books including *Mastering Cloud Computing* published by McGraw Hill, China Machine Press, and Morgan Kaufmann for Indian, Chinese and International Markets, respectively. He is one of the highly cited authors in computer science and software engineering worldwide.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:55 UTC from IEEE Xplore. Restrictions apply.